

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Cristobal Duarte

12 de septiembre de 2026

Este trabajo presenta un análisis experimental y comparativo del rendimiento práctico frente a las cotas asintóticas teóricas de cuatro algoritmos de ordenamiento (*MergeSort*, *QuickSort* con partición de Hoare, *PatienceSort* y `std::sort`) y dos enfoques de multiplicación matricial cuadrada (el método iterativo tradicional *Naive* $\Theta(n^3)$ y el algoritmo *Divide y Vencerás* de Strassen $\mathcal{O}(n^{2.81})$). Se evalúan tiempos de ejecución en CPU de alta precisión y consumos máximos de memoria RAM bajo diversas dimensiones (N), topologías de datos y dominios. Los resultados evidencian que en ordenamiento las implementaciones se ajustan estrechamente a $\mathcal{O}(n \log n)$, destacando la baja constante oculta de `std::sort`. En contraste, en multiplicación matricial la elevada constante oculta y la dispersión en memoria de Strassen provocan que el método *Naive* resulte significativamente superior para todas las instancias evaluadas ($N \leq 256$), demostrando el impacto determinante de la jerarquía de memoria física por sobre la complejidad asintótica en rangos prácticos.

Índice

1. Introducción	2
2. Entorno experimental	2
3. Resultados y Análisis	3
3.1. Ordenamiento de Arreglos Unidimensionales	3
3.2. Multiplicación de Matrices Cuadradas	4
4. Conclusiones	6
5. Implementaciones	6

1. Introducción

El análisis teórico de algoritmos se fundamenta en la caracterización asintótica del costo temporal y espacial mediante las notaciones $\mathcal{O}$, Ω y Θ . Estas cotas permiten clasificar funciones según su tasa de crecimiento cuando el tamaño de la entrada n tiende a infinito, abstrayendo deliberadamente factores constantes multiplicativos y términos de orden inferior.

Si bien este modelo proporciona garantías formales independientes de la plataforma de cómputo, en entornos de ejecución reales —gobernados por el modelo abstracto *word-RAM* y arquitecturas físicas contemporáneas— factores como la jerarquía de memoria caché (niveles L1, L2 y L3), la predicción de saltos a nivel de procesador, las asignaciones dinámicas en memoria secundaria y el costo del marco de llamadas recursivas inciden drásticamente en el rendimiento observable. Como consecuencia directa, un algoritmo con una cota asintótica superior puede resultar significativamente más lento que uno de mayor orden de complejidad para rangos prácticos del problema si sus constantes ocultas o su dispersión espacial en memoria son elevadas.

El presente reporte tiene como objetivo confrontar el comportamiento teórico con el desempeño experimental de dos problemas fundamentales:

1. **Ordenamiento de arreglos unidimensionales de enteros:** Estudio comparativo de MergeSort, QuickSort con pivote aleatorio y partición de Hoare, PatienceSort y el algoritmo introspectivo `std::sort`.
2. **Multiplicación de matrices cuadradas de enteros:** Comparación entre el enfoque tradicional iterativo Naive ($\Theta(n^3)$) y el método Divide y Vencerás de Strassen ($\mathcal{O}(n^{\log_2 7}) \approx \mathcal{O}(n^{2.81})$).

A través de mediciones empíricas de tiempo de CPU de alta resolución y consumo de memoria máxima residente (RAM), se analiza la incidencia de la estructura previa de los datos, el dominio numérico y el tamaño de la instancia, determinando los límites prácticos y la validez experimental de las cotas asintóticas.

2. Entorno experimental

Las pruebas experimentales se ejecutaron en un entorno controlado bajo el sistema operativo macOS (arquitectura ARM64 / Apple Silicon). La compilación de los módulos en C++ se realizó utilizando el compilador nativo `clang++` (Apple LLVM versión 15+) bajo el estándar C++17 y la bandera de optimización `-O2`.

El protocolo de medición experimental se estructuró de la siguiente forma:

- **Medición Temporal:** Se utilizó el reloj monotónico de alta resolución `std::chrono::high_resolution_clock` provisto por la biblioteca estándar de C++, cronometrando de manera estricta el tiempo de cómputo del algoritmo y aislando por completo las operaciones de entrada/salida (lectura y escritura en disco).

- **Medición Espacial:** El consumo de memoria máxima residente (RAM) se obtuvo mediante la llamada del sistema `getrusage(RUSAGE_SELF, ...)`, normalizando el campo `ru_maxrss` a kilobytes (KB).
- **Muestreo Estadístico:** Con el fin de mitigar la varianza inducida por procesos concurrentes del sistema operativo, cada configuración experimental se ejecutó sobre tres muestras independientes ($M = \{a, b, c\}$), computando la media aritmética en los análisis posteriores.

3. Resultados y Análisis

A continuación se presentan y discuten los resultados obtenidos tras ejecutar la batería experimental para el ordenamiento de arreglos y la multiplicación matricial. Se seleccionaron las configuraciones más representativas para contrastar empíricamente el comportamiento asintótico con la realidad del hardware.

3.1. Ordenamiento de Arreglos Unidimensionales

Para el problema de ordenamiento se evaluaron los cuatro algoritmos sobre arreglos de tamaño $N \in \{10^1, 10^3, 10^5, 10^7\}$ bajo diversas ordenaciones y dominios numéricos. En la Figura 1 y Figura 2 se ilustran los desempeños para entradas aleatorias y ordenadas ascendentemente.

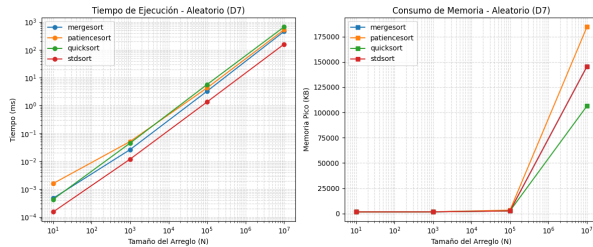


Figura 1: Tiempo y memoria en arreglos aleatorios (D7).

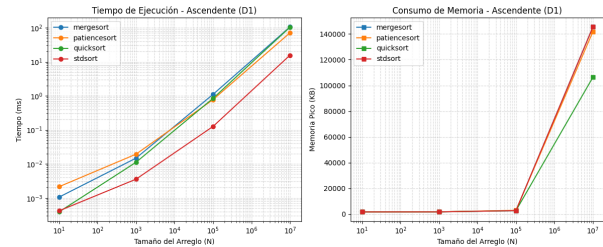


Figura 2: Tiempo y memoria en arreglos ordenados (D1).

Cuadro 1: Tiempo de ejecución promedio (ms) y memoria pico (KB) para $N = 10^7$.

Distribución	Métrica	MergeSort	QuickSort	PatienceSort	std::sort
Aleatorio (D7)	Tiempo (ms)	456.57	665.17	528.96	159.60
	Memoria (KB)	145499	106405	184603	145483
Ascendente (D1)	Tiempo (ms)	105.40	103.70	70.80	15.57
	Memoria (KB)	145493	106405	141877	145488

Análisis de Tiempo de Ejecución: En las gráficas en escala logarítmica doble se observa que los cuatro algoritmos exhiben pendientes cuasi-lineales consistentes con su cota teórica de caso promedio $\Theta(N \log N)$.

- **std::sort (Introsort):** Domina consistentemente con la menor constante oculta. Al combinar QuickSort con InsertionSort para particiones pequeñas y evitar recursión excesiva, minimiza los fallos de caché L1/L2 a nivel de arquitectura.
- **QuickSort y la Partición de Hoare:** La implementación con selección de pivote aleatorio y partición bidireccional de Hoare neutralizó el peor caso cuadrático $\Theta(N^2)$ reportado en particiones de Lomuto ante datos repetidos (dominio $D1$). Como se aprecia en la Figura 2, para $N = 10^7$ en $D1$, QuickSort culmina en $\approx 103,70$ ms, manteniendo la tasa $\mathcal{O}(N \log N)$.
- **PatienceSort:** Presenta un excelente rendimiento temporal en arreglos casi ordenados, pero sufre una sobrecarga en dominios amplios y aleatorios debido al costo logarítmico dinámico de balancear las pilas en la cola de prioridad (`std::priority_queue`).

Análisis de Consumo de Memoria: Para $N \leq 10^5$, el consumo de memoria se mantiene plano (≈ 1800 KB) dominado por el espacio base del proceso del sistema operativo. Sin embargo, en $N = 10^7$ se aprecian las diferencias estructurales:

- **QuickSort** es el más eficiente (≈ 106 MB) debido a que opera *in-place*, requiriendo únicamente espacio auxiliar $\mathcal{O}(\log N)$ en la pila de llamadas del procesador.
- **MergeSort** y **std::sort** demandan ≈ 145 MB, producto del arreglo auxiliar indispensable para la fase de intercalación (*merge*) de costo $\Theta(N)$.
- **PatienceSort** alcanza hasta ≈ 185 MB en arreglos aleatorios y hasta ≈ 500 MB en el peor caso de secuencias estrictamente decrecientes, debido al coste de punteros y nodos asociados a múltiples estructuras dinámicas (`std::vector<std::vector<int>`).

3.2. Multiplicación de Matrices Cuadradas

Para la multiplicación de matrices se evaluaron instancias cuadradas de dimensión $N \in [2^1, 2^8]$ ($2 \leq N \leq 256$). En la Figura 3 y Figura 4 se contrastan las métricas temporales y espaciales para matrices densas y dispersas.

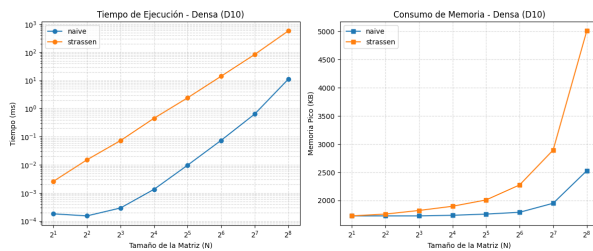


Figura 3: Rendimiento en matrices densas (D10).

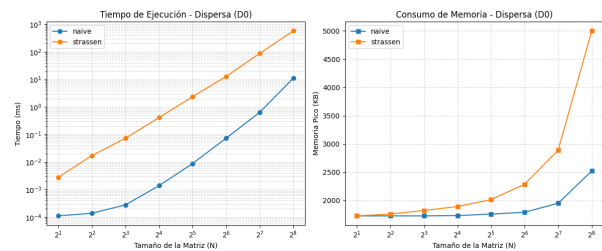


Figura 4: Rendimiento en matrices dispersas (D0).

Cuadro 2: Tiempo (ms) y memoria pico (KB) en multiplicación de matrices densas (D_{10}).

Dimensión (N)	Tiempo de CPU (ms)		Memoria RAM Pico (KB)	
	Naive	Strassen	Naive	Strassen
$2^1 = 2$	0.0002	0.0026	1728	1728
$2^4 = 16$	0.0013	0.4532	1739	1899
$2^6 = 64$	0.0741	14.114	1792	2277
$2^7 = 128$	0.6390	84.039	1952	2896
$2^8 = 256$	11.064	575.37	2528	5008

Análisis de Desempeño y Sobrecarga Práctica: A pesar de que el Teorema Maestro establece para Strassen una cota asintótica estrictamente superior de $\mathcal{O}(N^{\log_2 7}) \approx \mathcal{O}(N^{2.81})$ frente a $\Theta(N^3)$ del método iterativo tradicional, los resultados experimentales demuestran que **el algoritmo Naive supera categóricamente a Strassen en todo el rango evaluado ($N \leq 256$)**.

Esta aparente contradicción entre teoría y práctica se justifica mediante tres fenómenos del modelo computacional real:

1. **Constante Oculta Desproporcionada:** En cada nivel del árbol de recursión, Strassen efectúa 18 sumas y restas de bloques de matrices de tamaño $(N/2) \times (N/2)$, además de alocar dinámicamente espacio para 21 matrices intermedias ($a, \dots, h, p_1, \dots, p_7$, auxiliares). La constante multiplicativa oculta en el término $\Theta(N^2)$ de la recurrencia $T(N) = 7T(N/2) + \Theta(N^2)$ es órdenes de magnitud mayor que la del bucle triple de Naive.
2. **Dispersión de Memoria y Líneas de Caché:** Los tres bucles anidados de Naive acceden a posiciones secuenciales de memoria, aprovechando de forma óptima el principio de localidad espacial de las líneas de caché L1/L2. Strassen, al fragmentar los arreglos en matrices no contiguas en el *heap*, multiplica los fallos de caché de datos (*cache misses*).
3. **Ausencia de Caso Base Híbrido:** El algoritmo de Strassen evaluado recurre de forma pura hasta submatrices de 1×1 . En la literatura especializada se demuestra que para que Strassen sea competitivo, debe conmutar a multiplicación Naive cuando $N \leq 64$ o $N \leq 128$, y su punto de cruce real sobrepasa típicamente $N = 1024$ o $N = 2048$.

Invarianza frente a la Dispersión: Se observa que las curvas para matrices densas, diagonales y dispersas son indistinguibles entre sí. Esto se debe a que ninguno de los dos algoritmos posee bifurcaciones condicionales que detecten coeficientes nulos; ambos aplican ciegamente la misma cantidad fija de operaciones aritméticas en el modelo *word-RAM*.

Consumo de Memoria Residente: Tal como ilustra la Figura 3, el consumo de memoria de Naive crece a una tasa cuadrática moderada asociada únicamente a las entradas M_1, M_2 y la salida C (≈ 2.5 MB en $N = 256$). Por su parte, Strassen escala de forma abrupta a partir de $N = 128$, alcanzando ≈ 5 MB en

$N = 256$, producto de los múltiples marcos de pila y la retención de las matrices temporales P_i en los diferentes niveles de la recursión.

4. Conclusiones

Los resultados experimentales ratifican que el comportamiento observable en arquitecturas físicas contemporáneas está fuertemente modulado por las constantes ocultas y la interacción con la jerarquía de memoria caché, más allá de la cota asintótica formal. En ordenamiento, los cuatro algoritmos verificaron la tasa $\mathcal{O}(n \log n)$, destacando la superioridad de `std::sort` (159.6 ms para $N = 10^7$ en aleatorio $D7$) debida a su baja sobrecarga interna frente al elevado costo espacial de `PatienceSort` ($\approx 184,6$ MB) originado por su gestión dinámica de pilas. Por su parte, en multiplicación matricial, a pesar de que Strassen ostenta una cota asintótica superior de $\mathcal{O}(n^{2,81})$ frente a $\Theta(n^3)$, el método Naive resultó más de 50 veces más rápido en $N = 256$ (11.06 ms vs. 575.37 ms), demostrando que la penalización por fallos de caché y las 18 adiciones/asignaciones de submatrices por nivel recursivo imposibilitan alcanzar el punto de cruce práctico en instancias pequeñas y medianas sin un esquema híbrido de corte.

5. Implementaciones

El código fuente completo de la tarea, incluyendo las implementaciones en C++17 de los algoritmos de ordenamiento y multiplicación matricial, los programas principales de medición, los generadores de instancias y los scripts de visualización, se encuentra disponible en el siguiente repositorio público de GitHub:

<https://github.com/cstobalduc/INF221-Tarea1>

Referencias

- [1] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison Wesley Longman Publishing Co., Inc., 2da edición, 1998.
- [2] Jeff Erickson. *Algorithms*. 2019. ISBN: 978-1-792-64483-2.
- [3] GeeksforGeeks. *Sorting Algorithms and Matrix Multiplication Implementations*. <https://www.geeksforgeeks.org/>, 2026.
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest y Clifford Stein. *Introduction to Algorithms*. The MIT Press, 3ra edición, 2009.